17 Modern internals

This chapter covers

- Introducing JVM internals
- Reflection internals
- Method handles
- Invokedynamic
- Recent internal changes
- Unsafe

Java's virtual machine (JVM) is an extremely sophisticated runtime environment that has, for decades, prioritized stability and production-grade engineering. For these reasons, many Java developers have never needed to poke about in the internals, because it is simply not necessary to do so most of the time.

This chapter, on the other hand, is for the curious—the people who would like to know more, who would like to draw back the curtain and see some of the details of how the JVM is implemented. Let's start with method invocation.

17.1 Introducing JVM internals: Method invocation

To get going, let's look at a simple example, defined by the classes `Pet`, `Cat`, and `Bear` and the interface `Furry`. This can be seen in figure 17.1

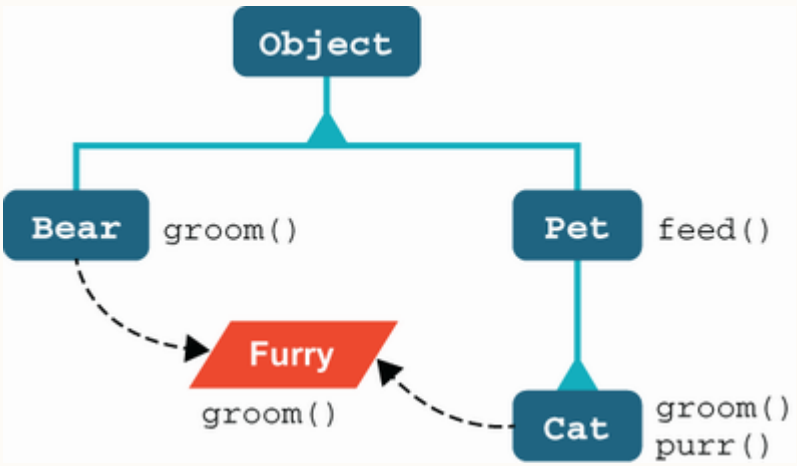


Figure 17.1 Simple inheritance hierarchy

We might also suppose that other subclasses of `Pet` exist (e.g., `Dog` and `Fish`) that aren't shown in the diagram to keep things clear. We'll use this example to explain in detail how the different invoke opcodes work, starting with `invokevirtual`.

7.1.1 Invoking virtual methods

The most common type of method invocation is calling an instance method on an object of a specific class (or a subclass) by use of an `invokevirtual` bytecode. This is known as a *dispatch* (i.e., call) of a *virtual method* (or just virtual dispatch), which means that the exact method to be invoked is determined at runtime, rather than compile time, by looking at the actual type of the object at runtime. When the JVM executes this bit of code:

```
Pet p = getPet();
p.feed();
```

the implementation of `feed()` that is actually called is determined at the point when the method needs to be executed.

The implementations can be different depending on whether `p` holds a `Cat` or a `Dog` (or a `Pet`, assuming the superclass is not abstract). It is also possible that `getPet()` returns an object of different subtypes of `Pet` at different times during the execution of the program. That doesn't matter—the implementation to be called is looked up each time the method is to be executed. Despite being a bit of a wall of text, this description is just how Java methods have always worked ever since you first learned the language.

Internally, to make this work, the JVM stores a table (per class), which holds the method definitions corresponding to that type, called the *vtable (this is what C++ programmers call a virtual function table).* This table is stored inside a special area of memory, called *metaspace*, inside the JVM that holds metadata that the VM needs.

NOTE In Java 7 and earlier, this metadata lived in an area of the Java heap called *permgen*.

To see how the vtable is used, we need to look briefly at the JVM metadata for a class. In Java, all objects live within the Java heap and are handled by reference only. HotSpot uses the general term *oop* (for "ordinary object pointer") to refer to the various internal data structures that live in the heap.

Every Java object must have an *object header*, which contains the following two types of metadata:

- Metadata that is specific to the particular instance of a class (the “mark word”)
- Metadata that is shared by all instances of a class (the “klass word”)

To save space, only one copy of the per-class metadata is stored, and each object that belongs to that class has a pointer to it—the klass word. In Figure 17.2 we can see a representation of a Java reference, held in a local variable, pointing at the start of a Java object header in the heap.

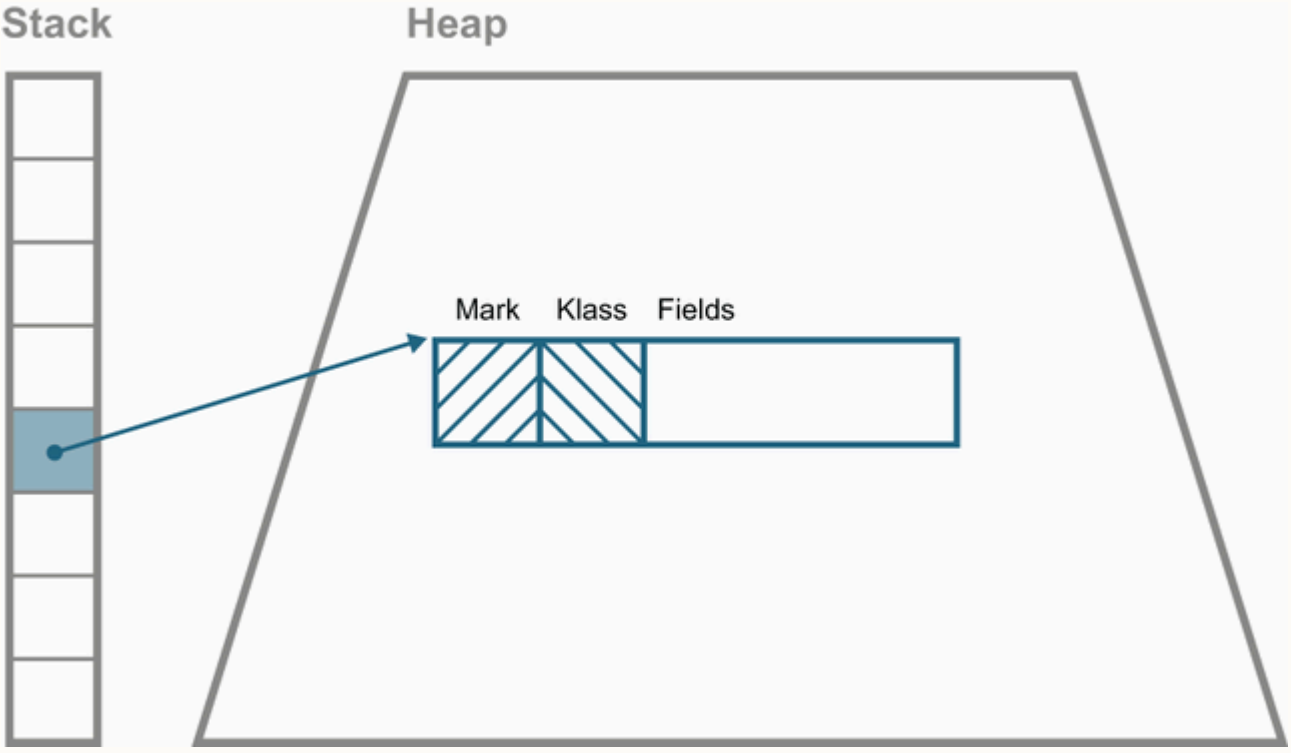


Figure 17.2 Java object header and layout

A *klass* is the JVM's internal representation of a Java class at runtime, stored in metaspace. It contains all of the information needed for the JVM to work with the class at runtime (e.g., method definitions and field layout).

Some of the information from the klass is available to the Java programmer via the `Class<?>` object corresponding to the type, but the klass and the `Class` are separate concepts. In particular, the klass contains information that is deliberately kept out of the reach of ordinary application code.

NOTE The choice of the spelling “klass” is quite deliberate, because it disambiguates the internal data structure from the other uses of the word “class” in written documentation but, sadly, not in spoken English. You may also see the word “clazz” or “clz” used —these usually name a Java variable that contains a `Class` object.

We can now explain virtual dispatch (implemented by the `invokevirtual` bytecode) in terms of the internal JVM structures, such as the klass, and, in particular, its vtable.

When the JVM encounters an `invokevirtual` instruction to execute, it pops the receiver object and any arguments for the method from the current method's evaluation stack.

NOTE A receiver object is the object upon which an instance method is being called.

The JVM object header layout starts with the mark word, with the klass word immediately following. So, to locate the method to be executed, the JVM follows the pointer into metaspace, where it consults the vtable of the klass to see exactly which code needs to be executed. This process can be seen in figure 17.3.

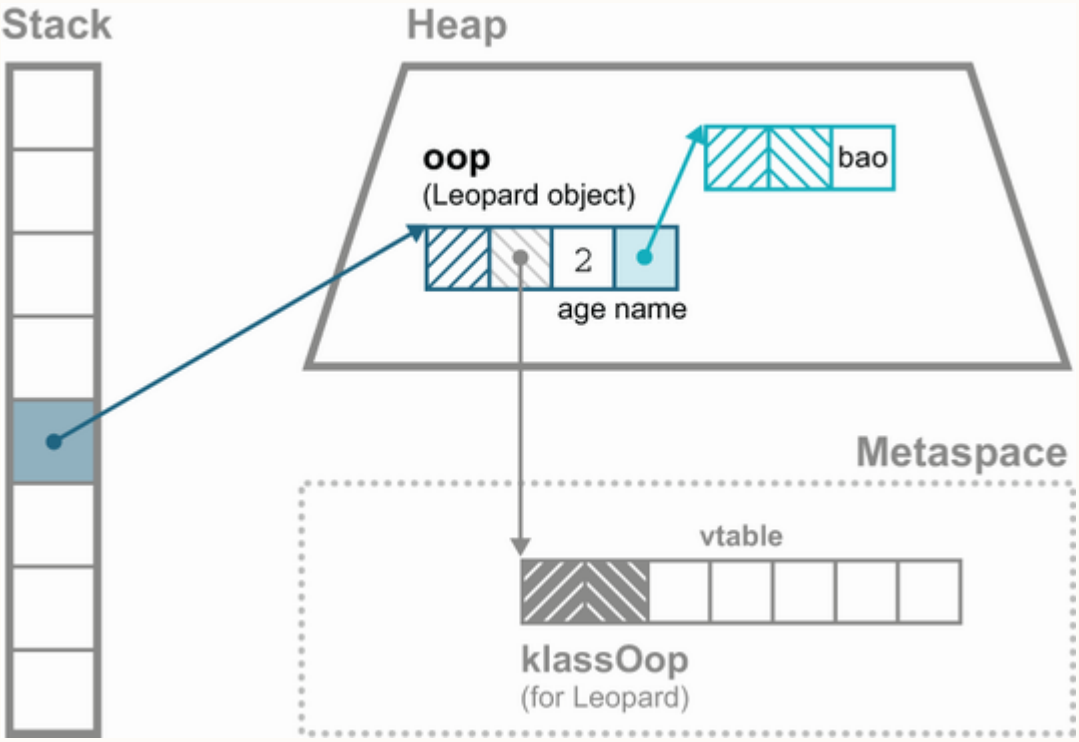


Figure 17.3 Locating a method implementation

If the klass does not have a definition for the method, the JVM follows a pointer to the klass corresponding to the direct superclass and tries again. This process is the basis of method overriding in the JVM.

To make it efficient, the vtables are laid out in a specific way. Each klass lays out its vtable so that the first methods to appear are those methods that the parent type defines. These methods are laid out in the exact order that the parent type used. The methods that are new to this type and are not declared by the parent type come at the end of the vtable.

When a subclass overrides a method, it will be at the same offset in the vtable as the implementation being overridden. This makes lookup of overridden methods completely trivial, because their offset in the vtable will be the same as their parent. In figure 17.4, we can see the vtable layout for some of the classes in our example.

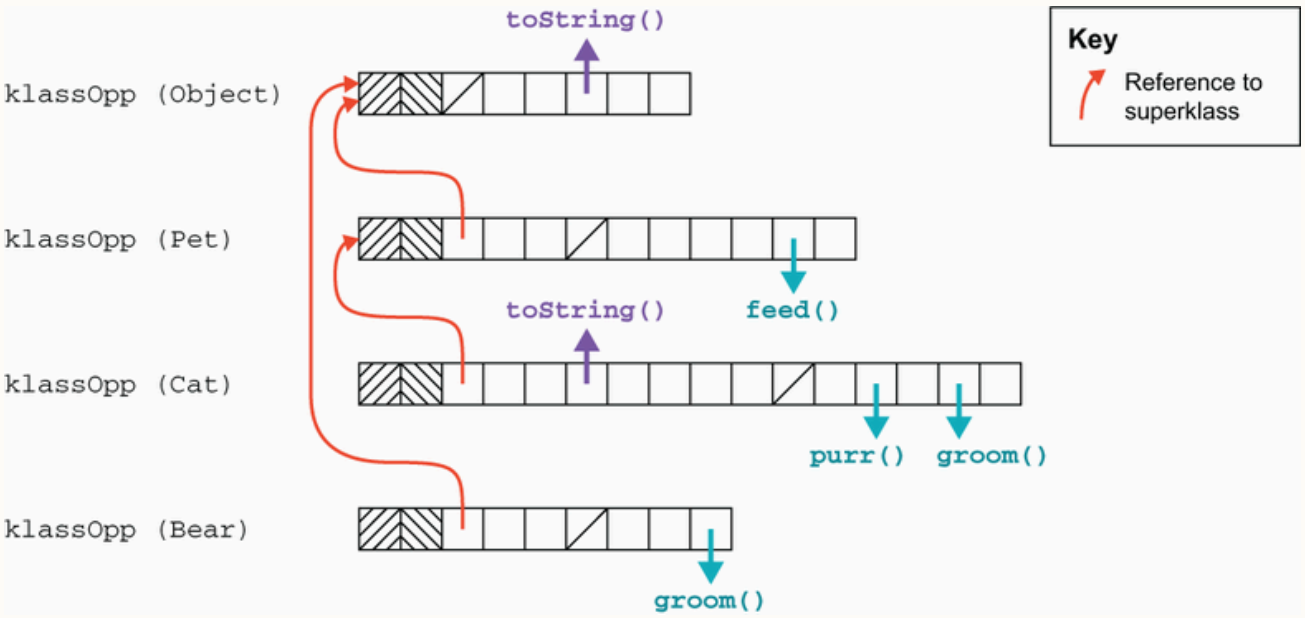


Figure 17.4 Vtable structure

So, if we call `Cat::feed`, the JVM will not find an override in the `Cat` class and will instead follow the superclass pointer to the klass of `Pet`. This does have an implementation for `feed()`, so this is the code that will be called.

NOTE This vtable structure—and efficient implementation of overriding—works well because Java implements only single inheritance of classes. There is only one direct superclass of any type (except `Object`, which has no superclass).

7.1.2 Invoking interface methods

In the case of `invokeinterface`, the situation is a little more complicated. For example, note that the `groom()` method will not necessarily appear in the same place in the vtable for every implementation of `Furry`. The different offsets for `Cat::groom` and `Bear::groom` are caused by the fact that their class inheritance hierarchies differ. The end result of this is that some additional lookup is needed when a method is invoked on an object for which only the interface type is known at compile time.

NOTE Even though slightly more work is done for the lookup of an interface call, you should not try to micro-optimize by avoiding interfaces. Remember that the JVM has a JIT compiler, and it will essentially eliminate the performance difference between the two cases.

Let's look at an example. Consider this bit of code:

```
Cat tom = new Cat();
Bear pooh = new Bear();
Furry f;
```

```
tom.groom();
pooh.groom();
f = tom;
f.groom();
f = pooh;
f.groom();
```

This produces the following bytecode:

```
0: new          #2              // class ch15/Cat
     3: dup
     4: invokespecial #3             // Method ch15/Cat."<init>":()V
     7: astore_1
     8: new          #4              // class ch15/Bear
    11: dup
    12: invokespecial #5             // Method ch15/Bear."<init>":()V
    15: astore_2
    16: aload_1
    17: invokevirtual #6             // Method ch15/Cat.groom:()V
    20: aload_2
    21: invokevirtual #7             // Method ch15/Bear.groom:()V
    24: aload_1
    25: astore_3
    26: aload_3
    27: invokeinterface #8,  1          // InterfaceMethod
                                // ch15/Furry.groom:()V
    32: aload_2
    33: astore_3
    34: aload_3
    35: invokeinterface #8,  1          // InterfaceMethod
                                // ch15/Furry.groom:()V
```

The two calls at 27 and 35 look like they are the same in the Java code but will actually invoke different methods, because the runtime contents of `f` is different. The call at 27 will actually invoke `Cat::groom`, whereas the call at 35 will invoke `Bear::groom`.

7.1.3 Invoking “special” methods

With this background of `invokevirtual` and `invokeinterface`, the behavior of `invokespecial` is now easy to understand. If a method is invoked by `invokespecial`, it does not undergo virtual lookup. Instead, the JVM will look only in the exact place in the exact vtable for the requested method.

An `invokespecial` is used for two cases: calls to a superclass method and calls to the constructor body (which is turned into a method called `<init>` in bytecode). In both of these cases, virtual lookup and the possibility of overriding are explicitly excluded.

We should mention two further corner cases that might seem to suggest the use of `invokespecial` (which is also called *exact dispatch*). The first is private methods—they cannot be overridden, and the exact method to be called is known when the class is compiled, so it might seem that they should be called via `invokespecial`. However, this situation is more complex than it appears. Let's look at an example to demonstrate:

```
public class ExamplePrivate {

  public void entry() {
    callThePrivate();
  }

  private void callThePrivate() {
    System.out.println("Private method");
  }
}
```

Let's compile this with Java 8 first. Decompiling it using `javap` gives this:

```
$ javap -c ch15/ExamplePrivate.class
Compiled from "ExamplePrivate.java"
public class ch15.ExamplePrivate {
  public ch15.ExamplePrivate();
    Code:
       0: aload_0
       1: invokespecial #1   // Method java/lang/Object."<init>":()V
       4: return

  public void entry();
    Code:
       0: aload_0
       1: invokespecial #2   // Method callThePrivate:()V
       4: return
}
```

Note that `javap` is being invoked without the `-p` switch, so the decompilation of the private method does not appear. So far, so good—the private method is indeed called via

`invokespecial`. However, if we recompile with Java 11 and look closely, we will see a different result, shown here:

```
$ javap -c ch15/ExamplePrivate.class
Compiled from "ExamplePrivate.java"
public class ch15.ExamplePrivate {
  public ch15.ExamplePrivate();
    Code:
       0: aload_0
       1: invokespecial #1 // Method java/lang/Object."<init>":()V
       4: return

  public void entry();
    Code:
       0: aload_0
       1: invokevirtual #2 // Method callThePrivate:()V    (1)
       4: return
}
```

(1) This is now invokevirtual.

As we can see, calls to private methods are handled differently in modern Java, which we will explain in section 17.5.3, when we meet nestmates.

7.1.4 Final methods

The other corner case is the use of `final` methods. At first glance, it might appear that calls to `final` methods would also be turned into `invokespecial` instructions—after all, they can't be overridden and the implementation that is to be called is known at compile time. However, the Java Language Specification has something to say about this case:

Changing a method that is declared `final` to no longer be declared `final` does not break compatibility with pre-existing binaries.

Suppose code in one class had a call to a `final` method in another class, which had been compiled into an `invokespecial`. Then, if the class with the `final` method was changed so that the method was made non-`final` (and recompiled), it could be overridden in a subclass.

Now suppose that an instance of the subclass was passed into the calling method in the first class. The `invokespecial` would be executed, and now the wrong implementation of the method would be called. This is a violation of the rules of Java's object orientation

(strictly speaking, it violates the Liskov Substitution Principle). For this reason, calls to `final` methods must be compiled into `invokevirtual` instructions.

NOTE In practice, HotSpot contains optimizations that allow the case of final methods to be detected and executed extremely efficiently.

We've introduced the basics of HotSpot's internals through the lens of virtual method dispatch. At this point, it might be interesting to reread the JIT compilation section of chapter 7—especially the sections on monomorphic dispatch and inlining. You may be able to gain a deeper understanding of these techniques now that you've seen a few of the details of how they are implemented.

17.2 Reflection internals

We met reflection in chapter 4 as a way of handling objects and calling methods dynamically at runtime. Now that we know about vtables, we can dig a bit deeper and see how reflection is implemented by the JVM.

Recall that we can obtain a `java.lang.reflect.Method` object from a class object and then invoke it, like this (eliding the exception handling):

```
Class<?> clazz = // ... some class
Method m = clazz.getMethod("toString");
Object ret = m.invoke(this);
System.out.println(ret);
```

But what does this `Method` object represent? It is really "the capability of calling a specific method dynamically at runtime." The dynamic nature of the call means that in the compiled code, we see only an `invokevirtual` of the `invoke()` method on `Method`, as shown next:

```
 0: ldc           #7   // ... some class
 2: astore_1
 3: aload_1
 4: ldc           #24  // String toString
 6: iconst_0                                    (1)
 7: anewarray     #26  // class java/lang/Class          (1)
10: invokevirtual #28  // Method java/lang/               (1)
                       // Class.getMethod:                (1)
                       // (Ljava/lang/String;[Ljava/lang/Class;)
                       // Ljava/lang/reflect/Method;
```

```
13: astore_2
14: aload_2
15: aload_0
16: iconst_0                                    ❷
17: anewarray     #2   // class java/lang/Object          ❷
20: invokevirtual #32  // Method java/lang/reflect/          ❷
                       // Method.invoke:                ❷
                       // (Ljava/lang/Object;[Ljava/lang/Object;)
                       // Ljava/lang/Object;
23: astore_3
```

❶ The call to getMethod() is variadic and is passed a size-0 array of Class objects.

❷ The call to invoke() is passed a size-0 array of Object objects (the arguments).

Note that there is no bytecode that refers to `toString()` via a method descriptor (such as `java/lang/Object.toString:()Ljava/lang/String;`)—only as the string `toString`.

Now, let's recall that class objects (e.g., `String.class`) are just regular Java objects—they have the properties of regular Java objects and are represented by oops. A class object contains a `Method` object for each method on the class, and these method objects are, again, just regular Java objects.

NOTE The `Method` objects are created lazily after class loading. You can sometimes see traces of this effect in an IDE's code debugger.

So how does the JVM actually implement reflection? Let's take a look at a bit of source for the `Method` class and see some of its fields:

```
    private Class<?>      clazz;                ❶
    private int           slot;                 ❷
    // This is guaranteed to be interned by the VM in the 1.4
    // reflection implementation
    private String        name;
    private Class<?>      returnType;
    private Class<?>[]    parameterTypes;
    private Class<?>[]    exceptionTypes;
    private int           modifiers;
    // Generics and annotations support
    private transient String    signature;
    // generic info repository; lazily initialized
    private transient MethodRepository genericInfo;
    private byte[]        annotations;
```

```
    private byte[]          parameterAnnotations;
    private byte[]          annotationDefault;
    private volatile MethodAccessor methodAccessor;      ❸
```

❶ The class this method belongs to

❷ The offset in the vtable where this methods lives

❸ The delegate that carries out the invocation

We already know that in Java, calling an instance method involves looking it up in a vtable. So, conceptually, we want to exploit the duality between the vtable and the array of `Method` objects held by the `Class` object. We can see this duality in figure 17.5 where the array of `Method` objects help by `Entry.class` is dual to the vtable on the klassOop for `Entry`.

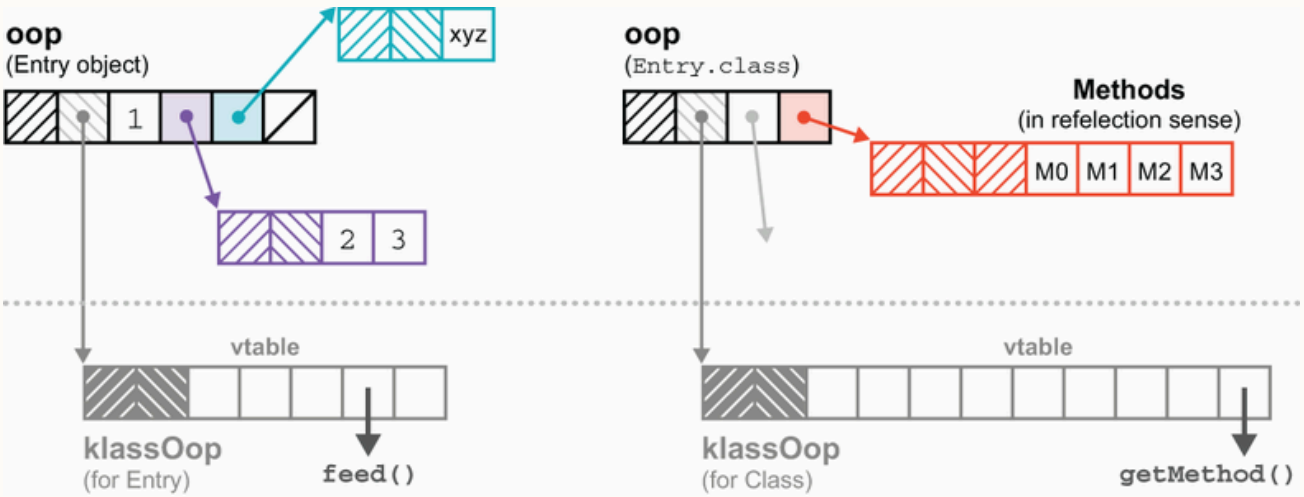


Figure 17.5 Reflection internals

Let's see how `Method` uses this duality to implement reflection. The key turns out to be the `MethodAccessor` object.

NOTE Some of the code that follows is simplified and based on an earlier version of Java, to aid understanding of the mechanism. The current shipping production code in Java 11 and later is more complex.

The `invoke()` method on `Method` looks a bit like this:

```
public Object invoke(Object obj, Object... args)
    throws IllegalAccessException, IllegalArgumentException,
        InvocationTargetException {
```

```
  if (!override) {                                    ❶
    if (!Reflection.quickCheckMemberAccess(clazz, modifiers)) {
      Class<?> caller = Reflection.getCallerClass();
      checkAccess(caller, clazz, obj, modifiers);
    }
  }
  MethodAccessor ma = methodAccessor;                 ❷
  if (ma == null) {
    ma = acquireMethodAccessor();
  }
  return ma.invoke(obj, args);                        ❸
}
```

❶ Performs a security access check (if not setAccessible())

❷ Volatile read of accessor

❸ Delegates to MethodAccessor

At the first reflective invocation of this method, `acquireMethodAccessor()` creates an instance of `DelegatingMethodAccessorImpl` that holds a reference to a `NativeMethodAccessorImpl`. These are classes defined in `sun.reflect` that both implement `MethodAccessor`. Note that they are not part of the API of the `java.base` module and cannot be called directly.

Here's `DelegatingMethodAccessorImpl` in full:

```
class DelegatingMethodAccessorImpl extends MethodAccessorImpl {
  private MethodAccessorImpl delegate;

  DelegatingMethodAccessorImpl(MethodAccessorImpl delegate) {
    setDelegate(delegate);
  }

  public Object invoke(Object obj, Object[] args)
      throws IllegalArgumentException, InvocationTargetException {
    return delegate.invoke(obj, args);
  }

  void setDelegate(MethodAccessorImpl delegate) {
    this.delegate = delegate;
  }
}
```

and here's `NativeMethodAccessorImpl`:

```
class NativeMethodAccessorImpl extends MethodAccessorImpl {
  private Method method;
  private DelegatingMethodAccessorImpl parent;
  private int numInvocations;

  // ...

  public Object invoke(Object obj, Object[] args)
        throws IllegalArgumentException, InvocationTargetException {

    if (++numInvocations >
        ReflectionFactory.inflationThreshold()) {       ❶
      MethodAccessorImpl acc = (MethodAccessorImpl)
        new MethodAccessorGenerator()
          .generateMethod(method.getDeclaringClass(),
                  method.getName(),
                  method.getParameterTypes(),
                  method.getReturnType(),
                  method.getExceptionTypes(),
                  method.getModifiers());       ❷
      parent.setDelegate(acc);                  ❸
    }

    return invoke0(method, obj, args);                ❹
  }

  private static native Object invoke0(Method m, Object obj, Object[] args);

  // ...
}
```

❶ Entered after an invocation threshold is reached

❷ Uses MethodAccessorGenerator to create a custom class that implements the reflective call

❸ Replaces the current object as a delegate with an instance of the new custom class

❹ If the threshold is not hit yet, proceeds with the native call

This technique—using a delegating accessor that can be patched with the new dynamically generated bytecode accessor—can be seen in figure 17.6. Note that the custom ac-

cessor class is a subclass of `MethodAccessorImpl` to allow the cast to succeed.

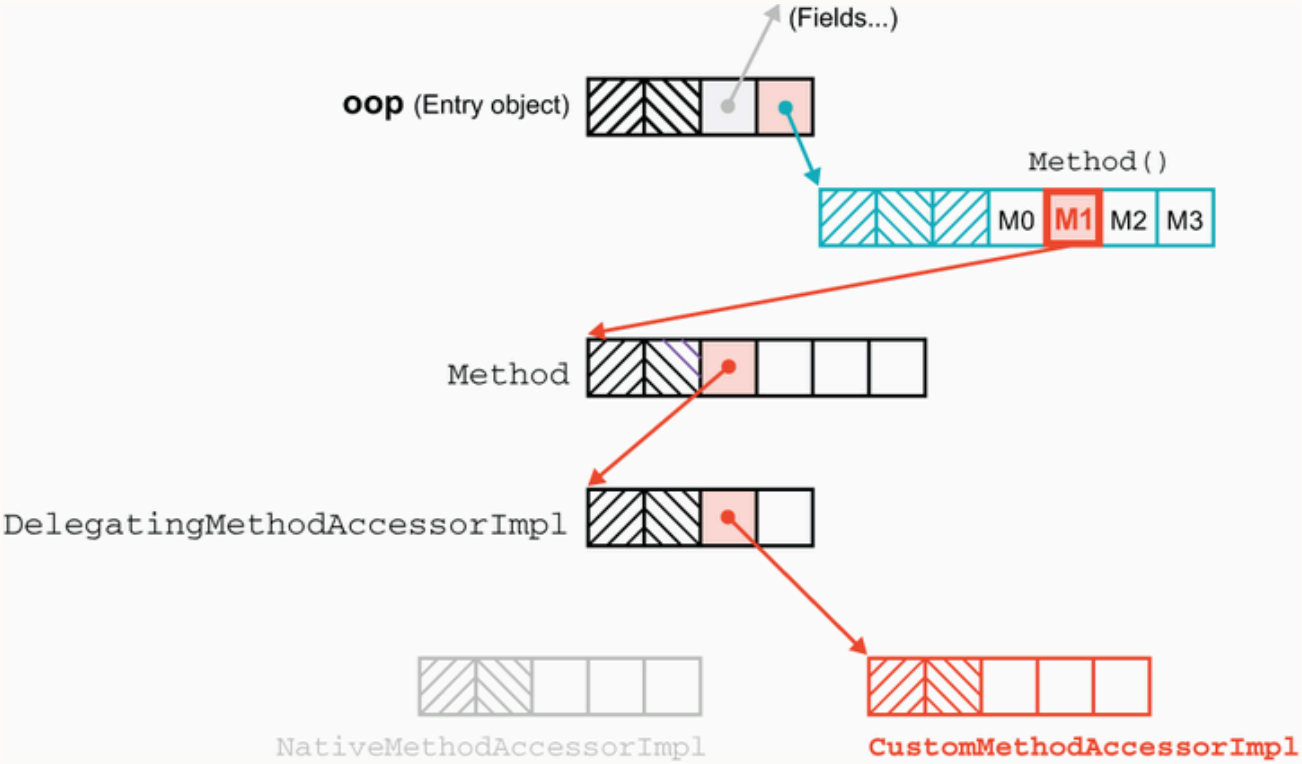


Figure 17.6 Implementing reflection

A word about performance: this mechanism involves trading off two different possible sources of slowness. On one hand, the native accessor uses a native call, which is slower than a Java method call and can't be JIT compiled. On the other, dynamically generating bytecode in `MethodAccessorGenerator` is potentially slow, and this can be a bad trade-off that we want to avoid for methods that are called by reflection only once. This trick, of lazily loading an accessor object and then dynamically patching the call site, is one that we will meet again, in a different guise, later in this chapter.

Also of note is that reflection also defeats inlining and the standard kinds of method dispatch that the JVM can optimize well. The call site for the method of `DelegatingMethodAccessorImpl` is said to be *megamorphic* (many possible implementations for the method) after patching because each instance of `Method` has a different, dynamically spun method accessor object. This means that some of the JVM's primary optimization mechanisms won't work well for reflective calls.

So the use of delegation and patching out of the native accessor is a compromise that aims to balance between acceptable performance for rarely called reflective methods while still preserving some of the benefits of JIT. This compromise, as well as the other problems of reflection, which we discussed back in chapter 4, led to a search for a better approach to the problems of dynamic invocation and lightweight method objects. In the next section, we introduce the first result of that research: the *Method Handles* API.

17.3 Method handles

The Method Handles API was added to Java with version 7. The core of this API is the package `java.lang.invoke`, and especially the class `MethodHandle`. Instances of this type represent the ability to call a method and are directly executable, in a similar way to `java.lang.reflect.Method` objects.

The API was produced as part of the project to bring `invokedynamic` (which we discuss in section 17.4) to the JVM. But method handle objects have applications in framework and regular user code that go far beyond the `invokedynamic` use cases.

We'll start by introducing the basic technology of method handles; then, we'll look at an extended example that compares them to some alternatives and summarizes the differences.

17.3.1 MethodHandle

What is a `MethodHandle`? The official answer is that it's a typed reference to a method that is directly executable. Another way of saying this is that a `MethodHandle` is an object that represents the ability to call a method safely.

NOTE A `MethodHandle` is similar, in many ways, to a `Method` object from `java.lang.reflect`, but the API is generally better, less cumbersome, and with several significant design flaws corrected.

There are two aspects to using method handles: obtaining them and using them. The second, using them, is very easy. Let's see a very simple example of calling a method handle. For now, we'll just assume that we have some static helper method `getTwoArgMH()` that returns a method handle that takes a receiver object `obj` and one call argument `arg0` and returns `String`.

Later, we'll explain how we'd get a method handle that matches that signature, but for now we just assume that we have a helper method that will create a method handle for us. The following usage should remind you of reflective calls:

```
MethodHandle mh = getTwoArgMH();                    (1)

try {
  String result = mh.invokeExact(obj, arg0);        (2)
} catch (Throwable e) {
```

```
    e.printStackTrace();
}
```

❶ Gets the method handle from a helper method

❷ Performs the call, passing a receiver and one argument

This looks like a reflective call to a method, as we saw in section 4.5.1—we're using `invokeExact()` on `MethodHandle` rather than `invoke()` on `Method`, but other than that, it should look pretty similar. However, this is possible only once we actually have a method handle object in the first place—so, how do we get one?

To get a handle for a method, we need to look it up via a *lookup context*. The usual way to get a context is to call the static helper method `MethodHandles.lookup()`. This will return a lookup context based on the currently executing method. From the lookup, we can obtain method handles by calling one of the `find*()` methods such as `findVirtual()` or `findConstructor()`.

The lookup context object can provide a method handle on any method that's visible from the execution context where the lookup was created. However, as well as the lookup context for methods, we also need to consider how to represent the signature of the method we want a handle to.

Recall the `Callable` interface from chapter 6. It represents a block of code to be executed, which is similar to method handles. However, one problem with `Callable` is that it can model only methods that take no arguments.

If we want to model all types of methods, we would have to create other interfaces, with increasing numbers of type parameters. We'd end up with a set of interfaces like this:

```
Function0<R>
Function1<R, P>
Function2<R, P1, P2>
Function3<R, P1, P2, P3>
...
```

This would very quickly lead to a huge proliferation of interfaces. This approach is taken by some non-Java languages (e.g., Scala), but not in Java.

It's also insightful to consider the way that Clojure does it. The `IFn` interface has `invoke` methods that represent all the different arities of functions (including a variadic form for functions that take more than 20 args). We met a simplified version of `IFn` in chapter 10.

However, Clojure is dynamically typed, so all `invoke` methods take `Object` for every parameter and also return `Object`—this eliminates all the complexity from handling generics. Clojure forms can also be written variadically quite naturally—Clojure will throw a runtime exception if a form is called with the wrong arity. Java uses neither of these approaches.

Instead, Java's method handles implement an approach that can model any method signature, without needing to produce a vast number of small classes. This is done by means of the new `MethodType` class.

7.3.2 MethodType

A `MethodType` is an immutable object that represents the type signature of a method. Every method handle has a `MethodType` instance that includes the return type and the argument types. But it doesn't include the name of the method or the receiver type—the type that an instance method is called on.

As one simple way to get new `MethodType` instances, you can use factory methods in the `MethodType` class. Here are a few examples:

```
var mtToString = MethodType.methodType(String.class);
var mtSetter = MethodType.methodType(void.class, Object.class);
var mtStringComparator = MethodType.methodType(int.class,
                                               String.class, String.class);
```

These are the `MethodType` instances that represent the type signatures of `toString()`, a setter method (for a member of type `Object`), and the `compareTo()` method defined by a `Comparator<String>`. The general instance follows the same pattern, with the return type passed in first, followed by the types of the arguments (all as `Class` objects), like this:

```
MethodType.methodType(RetType.class, Arg0Type.class, Arg1Type.class, ...);
```

As you can see, different method signatures can now be represented as normal instance objects, without the need to define a new type for each signature that was required. This also provides a simple way to ensure as much type safety as possible. If you want to know whether a candidate method handle can be called with a certain set of arguments, you can examine the `MethodType` belonging to the handle.

NOTE Passing around a single `MethodType` object is much more convenient than the cumbersome `Class[]` that reflection forces you to use.

Now that you've seen how `MethodType` objects solve the interface-proliferation problem, let's see how we can create method handles that point at methods from our types.

7.3.3 Looking up method handles

Let's take a look at how to get a method handle that points at the `toString()` method on the current class. Notice that we want `mtToString` to exactly match the signature of `toString()`—it has a return type of `String` and takes no arguments. The corresponding `MethodType` instance should be `MethodType.methodType(String.class)`, as shown here:

```
public MethodHandle getToStringMH() {
  MethodHandle mh;
  var mt = MethodType.methodType(String.class);
  var lk = MethodHandles.lookup();                    ❶

  try {
    mh = lk.findVirtual(getClass(), "toString", mt);  ❷
  } catch (NoSuchMethodException | IllegalAccessException mhx) {
    throw (AssertionError)new AssertionError().initCause(mhx);
  }

  return mh;
}
```

❶ Obtains the lookup context

❷ Looks up the handle from context

To get a method handle from a lookup object, you need to provide the class that holds the method you want, the name of the method, and a `MethodType` representing the appropriate signature. The method type is necessary to deal with overloaded methods.

It is very common to use a lookup context to find methods on the current class, but you can, in fact, use the context to get handles on methods belonging to *any* type, including JDK types. Of course, if you get handles from a class in a different package or module, the lookup context will be able to see only methods you have access to (e.g., public methods on public classes in exported packages). This is an important aspect of the method handles API: access control for a method handle is checked when the method is found, *not* when the handle is executed.

Once a method handle has been obtained, it's always safe to call it, because there are no further access control checks. A method handle can be created in one context, where access is allowed, and then passed to another context where access is *not* allowed, and it will still execute. This is an important difference from reflection.

NOTE Access control for invoking method handles cannot be circumvented, unlike reflective calls. There is no equivalent of the reflection `setAccessible()` hack we met in chapter 4.

Now that we have a method handle, the natural thing to do with it is to execute it. The API provides two main ways to do this: the `invokeExact()` and `invoke()` methods.

The `invokeExact()` method requires the types of arguments to exactly match what the underlying method expects. The `invoke()` method performs some transformations to try to get the types to match if they're not quite right (e.g., boxing or unboxing, as required).

After this introduction, we'll now move on to show a longer example of how method handles can be used to replace other techniques, such as reflection and small inner classes used to proxy capabilities.

7.3.4 Reflection vs. proxies vs. method handles

If you've spent any time dealing with a codebase that contains a lot of reflection, you're probably all too familiar with some of the pain that comes from reflective code. In this subsection, we want to show you how method handles can be used to replace a lot of reflective boilerplate and make your coding life a little easier.

To show the differences between method handles and other techniques, we've provided three ways to access the private `callThePrivate()` method from outside the class. There are two standard techniques: reflection and the use of an inner class acting as a proxy. We can compare these to a modern approach based on `MethodHandle`. An example of the three alternatives appears in the next listing.

Listing 17.1 Providing access three ways

```
public class ExamplePrivate {
  // Some state ...

  public void entry() {
    callThePrivate();
  }

  private void callThePrivate() {                    (1)
    System.out.println("Private method");
  }

  public Method makeReflective() {
    Method meth = null;

    try {
      Class<?>[] argTypes = new Class[] { Void.class };
      meth = ExamplePrivate.class
              .getDeclaredMethod("callThePrivate", argTypes);
      meth.setAccessible(true);
    } catch (IllegalArgumentException |
             NoSuchMethodException |
             SecurityException e) {
      throw (AssertionError)new AssertionError().initCause(e);
    }

    return meth;
  }

  public static class Proxy {
    private Proxy() {}

    public static void invoke(ExamplePrivate priv) {
      priv.callThePrivate();
    }
  }

  public MethodHandle makeMh() {
    MethodHandle mh;
    var desc = MethodType.methodType(void.class);          (2)

    try {
      mh = MethodHandles.lookup()
             .findVirtual(ExamplePrivate.class,
               "callThePrivate", desc);                  (3)
```

```
        } catch (NoSuchMethodException | IllegalAccessException e) {
            throw (AssertionError)new AssertionError().initCause(e);
        }

        return mh;
    }

}
```

❶ The private method we want to provide access to

❷ MethodType creation—we can use the exact type rather than needing to box here.

❸ The MethodHandle lookup

The example class provides three different capabilities that can access the private `callThePrivate()` method. In practice, only one of these capabilities would usually be provided—we're only showing all three to discuss the distinctions between them. In practice, as a user of an API, you should not need to care which approach is used.

In table 17.1, you can see that the main advantage of reflection is familiarity. Proxies may be easier to understand for simple use cases, but we believe method handles represent the best of both worlds. We strongly recommend their use in all new applications.

Table 17.1 Comparing Java's indirect method access technologies

Feature	Reflection	Inner class/lambda	Method handle
Access control	Must use `setAccessible()`. Can be disallowed by an active security manager.	Inner classes can access restricted methods.	Full access to all methods allowed from current context. No issue with security managers.
Type discipline	None. Ugly exception on mismatch.	Static. Can be too strict. May need a lot of metaspace for all proxies.	Typesafe at runtime. Doesn't consume much (if any) metaspace.
Performance	Slow compared to alternatives.	As fast as any other method call.	Aims to be as fast as other method calls.

One additional feature that method handles provide is the ability to determine the current class from a static context. If you've ever written logging code (such as for log4j) that looked like this:

```
Logger lgr = LoggerFactory.getLogger(MyClass.class);
```

then you know that this code is fragile. If it's refactored to move into a superclass or subclass, the explicit class name would cause problems. With method handles, however, you can write this:

```
Logger lgr = LoggerFactory.getLogger(MethodHandles.lookup().lookupClass());
```

In this code, the `lookupClass()` expression can be thought of as an equivalent to `getClass()`, which can be used in a static context. This is particularly useful in situations such as dealing with logging frameworks, which typically have a logger per class.

NOTE Method Handles has proved to be a very successful API. In fact, it's been so successful that in Java 18 (but not in 11 or 17), the implementing technology for reflection has been changed to rely on it, instead of the implementation we met in the last section.

With the technology of method handles in your toolbox of techniques, and armed with a working knowledge of bytecode from chapter 4, let's dive into the details of the `invokedynamic` opcode. It was introduced in Java 7 and (so far) is the only opcode ever to be added to the JVM's instruction set. The original use case of `invokedynamic` was to help non-Java languages get the most out of the JVM as a platform, but it has become a major agent of change within the platform, as we'll see.

17.4 Invokedynamic

This section deals with one of the most technically sophisticated new features of modern Java. But despite being enormously powerful, it isn't a feature that will necessarily be used directly by every working developer. Instead, this feature is mostly for frameworks developers and non-Java language implementors at present.

Therefore, it's OK to skip this section on first reading. To make best use of it, you will need to have read and understood the discussion of executing `invoke` instructions earlier in the chapter—it helps to know the rules that we are now about to break.

We'll cover the details of how `invokedynamic` works and look at some examples of decompiling a call site that makes use of the new bytecode. Note that it isn't necessary to fully understand this to use languages and frameworks that leverage `invokedynamic`, but this is the internals chapter, so we're going to get into the details.

As you might guess from the name, `invokedynamic` is a new type of invocation instruction, that is, it's used for making method calls. It's used to tell the JVM that it must defer figuring out which method to call until runtime.

This might not seem like much of a big deal—after all, `invokevirtual` and `invokeinterface` both decide at runtime which implementation is to be called. However, the target selection for those opcodes is subject to the constraints of the Java language inheritance rules and type system, so at least some *call target* information is known at compile time.

On the other hand, `invokedynamic` was created to relax these constraints, and it does so by calling a helper method (called a *bootstrap method*, or *BSM*) that makes the decision of which method ought to be called.

NOTE The target method (call target) for an `invokedynamic` site does not have to fit into the rules of Java's inheritance hierarchy at all—it is a user-defined choice.

To allow for this flexibility, `invokedynamic` opcodes refer to a special section of the constant pool of the class that contains extended entries to support the dynamic nature

of the call—the BSMs. These are a key part of `invokedynamic`, and all `invokedynamic` call sites have a corresponding constant pool entry for a BSM.

BSMs take in information about the call site and link the dynamic call. A BSM takes at least three arguments and returns a `CallSite` object. The standard arguments are of the following types:

- `MethodHandles.Lookup` —A lookup object on the class in which the call site occurs
- `String` —The name mentioned in the `NameAndType`
- `MethodType` —The resolved type descriptor of the `NameAndType`

Following these arguments are any additional arguments that are needed by the BSM. These are referred to as *additional static arguments* in the documentation. The returned call site holds a `MethodHandle`, which is the effect of calling the call site and will be executed as the actual invocation of the `invokedynamic`.

NOTE To allow the association of a BSM to a specific `invokedynamic` call site, a new constant pool entry type, also called `InvokeDynamic`, was added to the classfile format.

The call site of the `invokedynamic` instruction is said to be "unlaced" at class loading time. This means that no target method has been associated with the call site and will not be until the call site is reached (i.e., when the JVM tries to link, and then execute, that specific `invokedynamic` instruction).

At this point, the BSM will be called to determine what method should actually be called. BSMs always return a `CallSite` object (which contains a `MethodHandle`), and it will be "laced" into the call site. With the `CallSite` linked, the actual method call can then be made—it's to the `MethodHandle` being held by the `CallSite`.

In the simplest case, when a `ConstantCallSite` is used, once the lookup has been done once, it will not be repeated. Instead, the target of the call site will be directly called on all future invocations without any further work. It behaves in a similar way to a `CompletableFuture<CallSite>`. In practice, this means that the call site is now stable and is, therefore, friendly to other JVM subsystems, such as the JIT compiler. More complex choices, such as a `MutableCallSite` (or even a `VolatileCallSite`), are also possible, and these allow for the possibility of relinking a call site so that it can point at a different target method over time.

A non-constant call site can have many different method handles as its target over the lifetime of a program. In fact, being able to change the method called at a particular call site is a technique that can be important for non-Java languages.

For example, in JavaScript or Ruby, an individual object of a particular type can have methods defined on it that are not present on other instances of the class. This is not possible in Java—the class defines a set of methods that are used to construct the vtable when the class is loaded, and all instances share the same vtable. The use of `invokedynamic` with mutable call sites can allow for this non-Java feature to be implemented efficiently.

We should point out that you can't make `javac` emit `invokedynamic` from a regular method invocation—a Java method call is always turned into one of the four "regular" invoke opcodes that we met in chapter 4. Instead, Java frameworks and libraries (including those in the JDK) use `invokedynamic` for a variety of purposes. Lambdas provide a great case study of one of these purposes. Let's take a closer look.

7.4.1 Implementing lambda expressions

Lambdas have become ubiquitous in Java programming, but many Java programmers don't really know how they're implemented. Let's find out, starting with the following simple example:

```
public class LambdaExample {
    private static final String HELLO = "Hello World!";

    public static void main(String[] args) throws Exception {
        Runnable r = () -> System.out.println(HELLO);
        Thread t = new Thread(r);
        t.start();
        t.join();
    }
}
```

You might guess that the lambda is really just syntactic sugar for an anonymous implementation of `Runnable`. However, if we compile the previous class, we can see that only a single file `LambdaExample.class` is generated—there is no second class file (which is where the inner class would be placed, as we discussed in chapter 8). So, there is more to the story, as we'll see.

Instead, if we decompile, then we can see the lambda body has in fact been compiled into a private static method that appears in the main class:

```
private static void lambda$main$0();
    Code:
```

```
      0: getstatic     #7   // Field
                        // java/lang/System.out:Ljava/io/PrintStream;
      3: ldc           #9   // String Hello World!
      5: invokevirtual #10  // Method java/io/PrintStream.println:
                        // (Ljava/lang/String;)V
      8: return
```

and the main method looks like this:

```
  public static void main(java.lang.String[]) throws java.lang.Exception;
    Code:
      0: invokedynamic #2,  0  // InvokeDynamic #0:run:
                         // ()Ljava/lang/Runnable;
      5: astore_1
      6: new           #3      // class java/lang/Thread
      9: dup
     10: aload_1
     11: invokespecial #4      // Method java/lang/Thread."<init>"
                         // :(Ljava/lang/Runnable;)V
     14: astore_2
     15: aload_2
     16: invokevirtual #5      // Method java/lang/Thread.start:()V
     19: aload_2
     20: invokevirtual #6      // Method java/lang/Thread.join:()V
     23: return
```

The `invokedynamic` is acting as a call to an unusual form of factory method. The call returns an instance of some type that implements `Runnable`. The exact type is not specified in the bytecode, and it fundamentally does not matter. In fact, the actual returned type does not exist at compile time and will be created on demand at runtime.

We know that `invokedynamic` sites always have bootstrap methods associated with them. For our simple `Runnable` example, we have a single BSM in the appropriate section of the class file, as shown here:

```
  BootstrapMethods:
    0: #28 REF_invokeStatic java/lang/invoke/LambdaMetafactory.metafactory:
        (Ljava/lang/invoke/MethodHandles$Lookup;Ljava/lang/String;
         Ljava/lang/invoke/MethodType;Ljava/lang/invoke/MethodType;
         Ljava/lang/invoke/MethodHandle;Ljava/lang/invoke/MethodType;)
         Ljava/lang/invoke/CallSite;
      Method arguments:
```

```
    #29 ()V
    #30 REF_invokeStatic LambdaExample.lambda$main$0:()V
    #29 ()V
```

This is a bit hard to read, so let's decode it. The bootstrap method for this call site is entry #28 in the constant pool—an entry of type `MethodHandle`. It points at a static factory method `LambdaMetafactory.metafactory()` in the package `java.lang.invoke`. The metafactory method takes quite a few arguments, but they're mostly supplied by the additional static arguments belonging to the BSM (entries #29 and #30).

A single lambda expression generates three static arguments that are passed to the BSM: the lambda's signature, the method handle for the actual final invocation target of the lambda (i.e., the lambda body), and the erased form of the signature.

Let's follow the code into `java.lang.invoke` and see how the platform uses *metafactories* to dynamically create the classes that actually implement the target types for our lambda expressions. The BSM (i.e., the call to the `metafactory` method) returns a call site object, as always. When the `invokedynamic` instruction is executed, the method handle contained in the call site will return an instance of a class that implements the lambda's target type.

NOTE If the `invokedynamic` instruction is never executed, the dynamically created class will never be created.

The source code for the `metafactory` method is relatively simple, as shown next:

```
public static CallSite metafactory(MethodHandles.Lookup caller,
                                   String invokedName,
                                   MethodType invokedType,
                                   MethodType samMethodType,
                                   MethodHandle implMethod,
                                   MethodType instantiatedMethodType)
        throws LambdaConversionException {
    AbstractValidatingLambdaMetafactory mf;
    mf = new InnerClassLambdaMetafactory(caller, invokedType,
                                         invokedName, samMethodType,
                                         implMethod,
                                         instantiatedMethodType,
                                         false, EMPTY_CLASS_ARRAY,
                                         EMPTY_MT_ARRAY);
    mf.validateMetafactoryArgs();
```

```
        return mf.buildCallSite();
    }
```

The lookup object corresponds to the context where the `invokedynamic` instruction lives. In our case, that is the same class where the lambda was defined, so the lookup context will have the correct permissions to access the private method that the lambda body was compiled into.

The invoked name and type are provided by the JVM and are implementation details. The final three parameters are the additional static arguments from the BSM.

In the current implementation of lambdas, the metafactory delegates to code that uses an internal, shaded copy of the ASM bytecode libraries to spin up an inner class that implements the target type. This may change in the future.

Finally, we should note that it is perfectly possible to create a custom class that uses `invokedynamic` to do something special, but to construct such a class, you would have to use a bytecode manipulation library to produce a .class file with the `invokedynamic` instruction in it. One good choice for this is the ASM library(see **http://asm.ow2.org/**). We have mentioned this library a couple of times, and it's an industrial-strength library used in a wide range of well-known Java frameworks (including the JDK itself, as mentioned earlier). This marks the end of our discussion of `invokedynamic`, and it's time to move on to discuss some smaller, but still significant, changes to the internals.

17.5 Small internal changes

Sometimes small changes can have a big impact on a language. In this section, we're going to meet three small internal changes to the implementation that either help performance or correct an old piece of cruft in the platform. Let's start by talking about strings.

7.5.1 String concatenation

Remember that in Java, instances of `String` are effectively immutable. So, what happens when you concatenate two strings together with the `+` operator? The JVM must create a new `String` object, but there's more going on here than might be immediately apparent.

Consider a simple class with a `main()` method, as shown here:

```
public static void main(String[] args) {
  String str = "foo";
  if (args.length > 0) {
    str = args[0];
  }
  System.out.println("this is my string: " + str);
}
```

The Java 8 bytecode corresponding to this relatively simple method is as follows:

```
public static void main(java.lang.String[]);
Code:
  0: ldc #17          // String foo
  2: astore_2
  3: aload_1
  4: arraylength
  5: ifle 12                                          (1)
  8: aload_1
  9: iconst_0
 10: aaload
 11: astore_2
 12: getstatic #19     // Field java/lang/System.out:              (2)
                       // Ljava/io/PrintStream;                    (2)
 15: new #25           // class java/lang/StringBuilder            (3)
 18: dup                                                 (3)
 19: ldc #27           // String this is my string:                (3)
 21: invokespecial #29  // Method java/lang/                      (3)
                       // StringBuilder."<init>"                   (3)
                       // :(Ljava/lang/String;)V                   (3)
 24: aload_2
 25: invokevirtual #32  // Method java/lang/StringBuilder.append
                       // (Ljava/lang/String;)Ljava/lang/StringBuilder;
 28: invokevirtual #36  // Method java/lang/                      (4)
                       // StringBuilder.toString:                  (4)
                       // ()Ljava/lang/String;                     (4)
 31: invokevirtual #40  // Method java/io/                        (5)
                       // PrintStream.println:                     (5)
                       // (Ljava/lang/String;)V                    (5)
 34: return
```

(1) If the array is empty, jump forward to instruction 12.

(2) Loads System.out onto the stack

❸ Sets up StringBuilder

❹ Creates a string from StringBuilder

❺ Prints the string

We have a few things to notice in this bytecode. In particular, the appearance of `StringBuilder` may be a little surprising—we asked for concatenation of some strings, but the bytecode is telling us that we're really creating additional objects and then calling `append()`, then `toString()` on them.

Instructions 15–23 show the object creation pattern (`new`, `dup`, `invokespecial`) for the temporary `StringBuilder` object, but in this case, construction also includes an `ldc` (load constant) after the `dup`. This variant pattern indicates that you're calling a non-void constructor—`StringBuilder(String)`, in this case.

The reason behind all of this is that Java's strings are (effectively) immutable. We can't modify the string contents by concatenating it, so instead, we have to make a new object, and the `StringBuilder` is just one convenient way to do this.

The bytecode shape for Java 11 looks totally different, however:

```
public static void main(java.lang.String[]);
Code:
  0: ldc           #2      // String foo
  2: astore_1
  3: aload_0
  4: arraylength
  5: ifle          12
  8: aload_0
  9: iconst_0
 10: aaload
 11: astore_1
 12: getstatic     #3      // Field java/lang/System.out:
                           // Ljava/io/PrintStream;
 15: aload_1
 16: invokedynamic #4,  0  // InvokeDynamic #0:makeConcatWithConstants:
                           // (Ljava/lang/String;)Ljava/lang/String;
 21: invokevirtual #5      // Method java/io/PrintStream.println:
                           // (Ljava/lang/String;)V
 24: return
```

The first 12 instructions are identical to the Java 8 case, but then things start to change. One obvious change is that the `StringBuilder` temporary is completely gone. Instead, there's an `invokedynamic` at instruction 16. This, of course, requires a bootstrap method:

```
BootstrapMethods:
  0: #23 REF_invokeStatic java/lang/invoke/StringConcatFactory.
     makeConcatWithConstants:(Ljava/lang/invoke/MethodHandles$Lookup;
     Ljava/lang/String;Ljava/lang/invoke/MethodType;
     Ljava/lang/String;[Ljava/lang/Object;)Ljava/lang/invoke/CallSite;
    Method arguments:
     #24 this is my string: \u0001
```

This is a dynamic invocation of a static factory method `makeConcatWithConstants()` that lives on a class called `StringConcatFactory` in the package `java.lang.invoke`. This factory method takes in a string—the concatenation *recipe* for these specific arguments—and produces a `CallSite`, which is linked to a customized method for this specific case.

In general, this is machinery that is deep inside the JVM's implementation code. Most ordinary Java code would never call these methods directly and instead would rely on JDK code and libraries/frameworks that would call it.

NOTE The static argument to the bootstrap method includes the character `\u0001` (Unicode point 0001), which represents an ordinary argument to be interpolated into the concatenation recipe.

The `invokedynamic` callsites can be reused and the implementing classes can be dynamically created if need be. The implementations also have access to private APIs—such as zero-copy `String` constructors—that it would not be possible to expose as a part of `StringBuilder`.

7.5.2 Compact strings

When you first learned Java, you were introduced to the primitive types, and you learned that a `char` is two bytes in Java. It isn't too difficult to guess that under the hood, a Java string is implemented using a `char[]` to hold the individual characters of the string.

However, this is not quite true. It is true that in Java 8 and earlier the contents of a `String` are represented as a `char[]`. However, this representation causes an inefficiency that may not be obvious, so let's dig into it a little.

Java's two-byte `char` (which represents a UTF-16 character) is wasteful for any string that contains only characters from Western European languages, because the first byte of each `char` is always zero for such strings. This wastes almost 50% of the storage of the string for those languages, and they're very common.

To address this issue, Java 9 introduced a performance optimization: it allows a per-string choice of (currently) two representations. Each string can be encoded as either Latin-1 (for Western European languages) or UTF-16 (the original representation).

NOTE Latin-1 is also known by its standards number, ISO-8859-1. Try not to confuse it with ISO-8851, which is the international standard for determining the moisture content of butter.

Internally, the representation of a string has changed to a `byte[]`, with a string of `n` characters being represented in `n` bytes if the string is Latin-1 or `n * 2` bytes if not. The code in `java.lang.String` looks like this:

```
private final byte[] value;

/**
 * The identifier of the encoding used to encode the bytes in
 * {@code value}. The supported values in this implementation are
 *
 * LATIN1
 * UTF16
 *
 * @implNote This field is trusted by the VM, and is a subject to
 * constant folding if String instance is constant. Overwriting this
 * field after construction will cause problems.
 */
private final byte coder;

static final byte LATIN1 = 0;
static final byte UTF16  = 1;
```

Depending on the nature of the workload, this can cause significant savings in the common Latin-1 case. On the other hand, applications that deal primarily with text in, for example, one of the CJKV languages (Chinese, Japanese, Korean, and Vietnamese) will not see any space savings from this internal change.

In the field, applications using Western languages can see up to 30% or even 40% heap size savings when moving from Java 8 to 11, just from this change alone. Smaller heap

sizes means smaller containers, and that can translate to a visible saving in the costs of cloud compute for your applications.

To conclude this discussion, a quick note about the performance impact of this change: it provides a great example of why we must treat performance as an experimental science and measure from the top down. The introduction of the two different `String` representations does involve more code being executed because string operations now need to have two separate implementations—one for Latin-1 and one for UTF-16—and so code needs to inspect `coder` and branch based on the result.

However, the key performance question is, "Does the extra code matter?" that is, do the benefits outweigh the cost of this additional "complexity tax"? The benefits of the change include the following:

- Smaller heap sizes
- Potentially faster GC times
- Better cache locality for Latin-1 strings

We can also question how much the additional comparison and branch operations really affect the amount of code actually executed—remember that the JIT compiler does a lot of nonobvious optimizations and can exploit the time spent waiting for cache misses, so the extra instructions needed by compact strings may actually come for free, essentially.

NOTE In general, "counting instructions executed" is not a good way to reason about Java performance.

This picture of countervailing forces and trade-offs, the impact of which can be determined only by measurement of large-scale observables, is precisely the performance model that we discussed in chapter 7. In this specific case, the smaller heap size may translate directly into reduced cloud hosting costs because smaller containers can be used.

7.5.3 Nestmates

Nestmates were specified in JEP 181: Nest-Based Access Control, and the change essentially corrects an implementation hack that dates back all the way to Java 1.1 to do with inner classes. Let's look at an example and see the changes in bytecode that have been made to support nestmates:

```
public class Outer {
    private int i = 0;
```

```
    public class Inner {
      public int i() {
        return i;
      }
    }
}
```

If we compile this code, we will end up with two separate class files, whether we compile with Java 8 or 17. However, the bytecode is different in each case. We can use `javap` to look at the difference between the two cases. Here's Java 8:

```
Compiled from "Outer.java"
public class Outer {
  private int i;

  public Outer();
    Code:
      0: aload_0
      1: invokespecial #2      // Method java/lang/Object."<init>":()V
      4: aload_0
      5: iconst_0
      6: putfield      #1      // Field i:I
      9: return

  static int access$000(Outer);          (1)
    Code:
      0: aload_0
      1: getfield      #1      // Field i:I
      4: ireturn
}
```

(1) The compiler has inserted this "bridge" method.

with the separate class file for the inner class:

```
Compiled from "Outer.java"
public class Outer$Inner {
  final Outer this$0;

  public Outer$Inner(Outer);
    Code:
      0: aload_0
```

```
      1: aload_1
      2: putfield      #1    // Field this$0:LOuter;
      5: aload_0
      6: invokespecial #2    // Method java/lang/Object."<init>":()V
      9: return

  public int i();
    Code:
      0: aload_0
      1: getfield      #1    // Field this$0:LOuter;
      4: invokestatic  #3    // Method                  (1)
                             // Outer.access$000:(LOuter;)I
      7: ireturn
}
```

(1) The bridge method used here

Note how the *synthetic* access method (or bridge method) `access$000()` has been added to the outer class to provide package-private access to the private field that we access in the inner class. Now let's see what happens to the bytecode if we recompile the source under Java 17 (or 11):

```
Compiled from "Outer.java"
public class Outer {
  private int i;

  public Outer();
    Code:
      0: aload_0
      1: invokespecial #2        // Method java/lang/Object."<init>":()V
      4: aload_0
      5: iconst_0
      6: putfield      #1        // Field i:I
      9: return
}
```

The synthetic accessor has completely disappeared. Instead, look at the inner class, shown here:

```
Compiled from "Outer.java"
public class Outer$Inner {
  final Outer this$0;
```

```
  public Outer$Inner(Outer);
    Code:
      0: aload_0
      1: aload_1
      2: putfield      #1      // Field this$0:LOuter;
      5: aload_0
      6: invokespecial #2      // Method java/lang/Object."<init>":()V
      9: return

  public int i();
    Code:
      0: aload_0
      1: getfield      #1      // Field this$0:LOuter;
      4: getfield      #3      // Field Outer.i:I
      7: ireturn
}
```

This is now direct access of a private field, as shown next:

```
SourceFile: "Outer.java"
NestMembers:
  Outer$Inner
InnerClasses:
  public #6= #5 of #3;          // Inner=class Outer$Inner of class Outer
```

Java 11 introduced the concept of *nests*, which are actually a generalization of the existing concept of nested classes. In previous versions of Java, to share the same access control context, the source code for one class had to be physically located inside the source code of another class. In the new concept, a group of class files can form a nest, in which *nestmates* share a common access control mechanism and have unrestricted direct (and reflective) access to each other—including to private members.

NOTE The arrival of nestmates has subtly altered the meaning of `private`, as we saw earlier when we discussed `invokespecial`.

This change is indeed small, but it not only removes some less-than-perfect implementation cruft but is also needed for forthcoming changes in the platform. Let's move on from the small changes and discuss one of the major (and most notorious?) aspects of JVM internals: the `Unsafe` class.

17.6 Unsafe

In the Java platform, if some feature or behavior seems "magical," it is normally accomplished by using one of three primary mechanisms: reflection, class loading (including associated bytecode transformation), or `Unsafe`.

A power user of Java will seek to understand all three of these techniques, even if they resort to them only when necessary. The principle that "just because you *can* do something, it does not mean that you *should* " applies to our design choices in software as much as it does elsewhere.

Of the three, `Unsafe` is the most potentially dangerous (and powerful) because it provides a way to do certain things that are otherwise impossible and that break well-established rules of the platform. For example, `Unsafe` allows Java code to do the following:

- Directly access hardware CPU features
- Create an object but not run its constructor
- Create a truly anonymous class without the usual verification
- Manually manage off-heap memory
- Perform many other "impossible" things

The Java 8 `Unsafe` class, `sun.misc.Unsafe`, warns us of its very nature immediately—not only in the class name but also by the package in which it lives. The `sun.misc` package is an internal, implementation-specific location and not something that Java code should ever touch directly.

NOTE In Java 9 and later versions, the danger of `Unsafe` is made even clearer, as the functionality has been moved to a module called `jdk.unsupported`.

Java libraries are, of course, not supposed to couple directly to these types of implementation details. Reinforcing this standpoint, the attitude of Java's platform maintainers has long been that end users who break the rules and link to internal details do so at their own risk.

However, the inconvenient truth is that this API, unsupported as it is, has widespread usage by library authors. It is not an official standard for Java, but it had become a dumping ground for nonstandard but necessary platform features of varying safety.

To illustrate this, let's take a look at a classic use of `Unsafe`: the use of the hardware feature known as "compare and swap," or CAS. This capability is present on virtually all modern CPUs but famously is not a part of Java's memory model (the JMM).

In our example, we'll recall the `Account` class we met back in chapter 5. For technical reasons, we'll assume the balance is an `int` rather than a `double` for this section. The `Account` interface is defined as follows:

```
public interface Account {
    boolean withdraw(int amount);

    void deposit(int amount);

    int getBalance();

    boolean transferTo(Account other, int amount);
}
```

We'll implement it in two different ways. First, we'll stick to the rules and use synchronization. Two of the methods on the interface look like this in `SynchronizedAccount`:

```
public class SynchronizedAccount implements Account {
    private int balance;

    public SynchronizedAccount(int openingBalance) {
        balance = openingBalance;
    }

    @Override
    public int getBalance() {
        synchronized (this) {
            return balance;
        }
    }

    @Override
    public void deposit(int amount) {
        // Check to see amount > 0, throw if not
        synchronized (this) {
            balance = balance + amount;
        }
    }
```

Now let's compare this to the atomic, `Unsafe` implementation, which contains significantly more boilerplate, due to having to access the `Unsafe` class reflectively:

```
public class AtomicAccount implements Account {
   private static final Unsafe unsafe;                          ❶
   private static final long balanceOffset;                     ❷

   private volatile int balance = 0;                            ❸

   static {
      try {
         Field f = Unsafe.class.getDeclaredField("theUnsafe");
         f.setAccessible(true);
         unsafe = (Unsafe) f.get(null);                         ❹
         balanceOffset = unsafe.objectFieldOffset(              ❺
                        AtomicAccount.class
                         .getDeclaredField("balance"));
      } catch (Exception ex) { throw new Error(ex); }
   }

   public AtomicAccount(int openingBalance) {
      balance = openingBalance;
   }

   @Override
   public double getBalance() {
      return balance;                                           ❻
   }

   @Override
   public void deposit(int amount) {
      // Check to see amount > 0, throw if not
      unsafe.getAndAddInt(this,
                     balanceOffset, amount);                    ❼
   }

   // ...
```

❶ Our copy of the Unsafe object

❷ The numeric value of the pointer offset for the balance field relative to the object start

❸ The actual balance field

❹ Looks up the Unsafe object reflectively

❺ Computes the pointer offset

❻ A volatile read of balance—no locks

❼ Updates the balance using CAS operations

In this example, we are doing several things that are supposedly impossible in Java. First, we are computing a pointer offset (the offset where the field `value` lives relative to the start of `AtomicAccount` objects). There is no sequence of JVM bytecode instructions that can provide this—only native code that directly accesses the JVM's internal data structures can do it. The method `objectFieldOffset()` on the `Unsafe` object enables us to do that.

Second, we are performing a lock-free, atomic add on the balance. This is not possible within the terms of the JMM because `volatile` grants us only one operation (a read *or* a write) but addition requires both a read *and* a write. Let's look at the code in the `getAndAddInt()` method in `Unsafe` to see how this is done:

```
public final int getAndAddInt(Object o, long offset, int delta) {
   int v;
   do {
      v = getIntVolatile(o, offset);                ❶
   } while (!compareAndSetInt(o, offset, v, v + delta));   ❷
   return v;
}
```

❶ Programmatic volatile access

❷ A low-level CAS operation

In this code, we are choosing the memory access mode (in this case, `volatile`) instead of having that determined by how the variable was declared. We are also directly accessing memory via pointer offset, not by field indirection—another action that is impossible in normal Java.

NOTE The implementation in JDK 11+ uses an "internal `Unsafe`" object, for encapsulation reasons—it is this code that's shown here.

The semantics of the CAS method follows: on an object `o`, which has a field at a given `offset` from the start of the object header, as a single CPU operation:

1. Compare the current state of the memory location (four bytes) to an int `v`.
2. If the value of `v` matches, update it to `v + delta`.
3. Return `true` if the replacement succeeded; return `false` if it failed.

The replacement might fail because a thread that is actively running on another CPU has updated the memory location between the volatile read and the CAS. In this case, the method `compareAndSetInt()` returns `false`, and the `do-while` loop sends us around for another try.

So, these types of operations are lock-free but *not* loop-free. Highly contended fields that are being operated on by many threads may require us to spin around the loop for some time before the atomic addition eventually succeeds, but there is no possibility of Lost Update.

NOTE If we trace down into the code in the JDK, we can see that this implementation is pretty close to what the JDK actually does for `AtomicInteger`.

For the sake of completeness, let's have a look at the `withdraw()` implementation as well:

```
@Override
public boolean withdraw(int amount) {
  // Check to see amount > 0, throw if not
  var currBal = balance;                          ❶
  var newBal = currBal - amount;
  if (newBal >= 0) {
    if (unsafe.compareAndSwapInt(this,            ❷
                    balanceOffset,
                    currBal, newBal)) {
      return true;
    }
  }
  return false;
}
```

❶ A volatile read of balance

❷ Attempts to update balance via a low-level CAS

This case is a little bit different, because we are directly using the low-level API for the balance update. This is necessary, because we must maintain the constraint that the ac-

count balance must not become negative. This requires extra operations, such as the comparison on `newBal`.

For the case of a deposit, we can use the higher-level API, which loops until it succeeds. However, this is because money can always be deposited into an account, regardless of its state. If we used the same technique here, this withdrawal could spin indefinitely because there may not be enough money in the account to satisfy it.

Instead, we take the approach that we try once and fail the withdrawal if the CAS operation fails. This removes the race condition of two withdrawal operations attempting to claim the same funds but has the side effect that a withdrawal that *should* succeed can be spuriously failed because of a deposit occurring on another thread (which alters the balance after the volatile read).

NOTE We could introduce a `for` loop to the withdrawal code to reduce the chance of spurious failures, but it must be a provably finite loop.

In benchmarks, the performance difference between these two approaches is quite considerable—the `Unsafe` implementation is roughly a factor of two to three times faster on modern hardware. However, you should not use techniques like this in your end user code. As already discussed, many (virtually all?) modern frameworks already use `Unsafe`. There is unlikely to be any performance benefit whatsoever in coding directly against `Unsafe` rather than using what your framework of choice provides.

More important, you are breaking the rules of the Java specification by doing this: using internal capabilities that do not necessarily follow the rules in the way that user code is supposed to. In the next section, we will discuss how recent versions of Java have tried to reduce this unsupported API and replace it with fully supported alternatives.

17.7 Replacing Unsafe with supported APIs

Recall that in chapter 2, we met Java modules. This encapsulation mechanism provides a strict exporting capability and removes the ability to call code in internal packages. How does this affect `Unsafe` and the code that uses it?

Given the number of frameworks and libraries that depend upon `Unsafe`, they would be unable to upgrade to versions of Java that do not permit reflective access to `Unsafe`.

NOTE The `Unsafe` object must be accessed reflectively because the platform already prevents direct access to it for non-JDK code.

In Java 11+, the modules system provides the `jdk.unsupported` module. It is declared like this:

```
module jdk.unsupported {
    exports sun.misc;
    exports sun.reflect;
    exports com.sun.nio.file;

    opens sun.misc;
    opens sun.reflect;
}
```

This code provides access for any application that explicitly depends upon the `unsupported` module and—crucially—also provides unrestricted reflective access to `sun.misc`, the package containing `Unsafe`. Although this helps move `Unsafe` into a more module-friendly form, we may legitimately ask: for how long should this compromise access be maintained?

This is giving `Unsafe` a temporary pass for a short time only—the real solution is for the Java platform team to create new, supported APIs that can replace the "safe" features of `sun.misc.Unsafe` and then remove or close the `jdk.unsupported` module once Java library authors have had a chance to migrate to the new APIs.

NOTE The closing of `Unsafe` affects everyone using a *very* wide range of frameworks—it's not an exaggeration to say that basically every nontrivial application in the Java ecosystem relies upon `Unsafe` indirectly in one way or another.

One of the major `Unsafe` APIs that needs to be removed is programmatic access modes for memory, such as `getIntVolatile()`. The replacement is the VarHandles API, which is the subject of our next section.

7.7.1 VarHandles

The VarHandles API, introduced in Java 9, looks to extend the Method Handles concept to provide similar functionality for fields and memory access. Recall that, as discussed in chapter 5, in the JMM, only two memory access modes are provided: normal access and volatile ("reread from main memory, disregarding CPU caches and stalling until read completes"). Not only that, but the Java language provides a way to express these modes only at the field level. All accesses are done in normal mode, unless a field is explicitly declared `volatile`, and in that case, *all* access to that field is done in volatile mode. What if these provisions are insufficient for modern applications?

NOTE Volatile is a Java language fiction. Memory is just memory, and there are not separate banks of volatile-access and non-volatile-access memory chips.

One important goal of VarHandles is to allow new ways of accessing memory, that is, to provide a supported, and superior, alternative to `Unsafe` usage, such as alternative ways to perform CAS or general volatile access.

To see this in action, let's look at a quick example that shows how we might approach using VarHandles to replace `Unsafe` in the account class:

```
public class VHAccount implements Account {
   private static final VarHandle vh;
   private volatile int balance = 0;

   static {
      try {
         var l = MethodHandles.lookup();                    (1)
         vh = l.findVarHandle(VHAccount.class,              (2)
                              "balance", int.class);
      } catch (Exception ex) { throw new Error(ex); }
   }

   @Override
   public void deposit(int amount) {
      // Check to see amount > 0, throw if not
      vh.getAndAdd(this, amount);                           (3)
   }

   // ...
}
```

(1) Creates a Lookup object

(2) Obtains a VarHandle for the balance, and caches it

(3) Uses the VarHandle to access the field, using volatile memory semantics

This is functionally equivalent to the version that uses `Unsafe`, but it now uses only fully supported APIs.

The use of `MethodHandles.Lookup` is an important change. Unlike reflection, which relies upon `setAccessible()` to access private fields, the lookup object has whatever per-

missions that the calling context has, which includes access to the private field `balance`.

The migration away from reflection and toward method and field handles means that a number of methods that were present in `Unsafe` in Java 8 can now be removed from the unsupported API, including the following:

- `compareAndSwapInt()`
- `compareAndSwapLong()`
- `compareAndSwapObject()`

The equivalents of these methods are found on `VarHandle`, along with useful accessor methods. There are also get and put methods for the primitive types and object, in both normal and volatile access modes, as well as methods for building efficient adders, such as:

- `getAndAddInt()`
- `getAndAddLong()`
- `getAndSetInt()`
- `getAndSetLong()`
- `getAndSetObject()`

Another key goal of VarHandles is to allow low-level access to the new memory order modes available in JDK 9 and later. These new concurrency barrier modes for Java 9 also require some rather modest updates to the JMM.

Overall, definite progress has been made in creating alternatives to the de facto APIs of `Unsafe`. For example, in addition to VarHandles, the `getCallerClass()` functionality from `Unsafe` is now available in the stack-walking API defined by JEP 259 (see **https://openjdk.java.net/jeps/259**). However, there is still more to do.

7.7.2 Hidden classes

Hidden classes are described in JEP 371 (see **https://openjdk.java.net/jeps/371**). This internal feature is designed for platform and framework authors. The JEP aims to provide a supported API for one of the most common usages of `Unsafe`: the desire to create on-the-fly classes that cannot be used directly by other classes (but can be handled indirectly).

These classes have sometimes been referred to as *anonymous classes*, and the method in `Unsafe` is called `defineAnonymousClass()`. However, that term is confusing to develop-

ers, because in the context of normal Java application code, it means a nested implementation of some interface that declares its static type to be the interface, like this:

```
public class Scratch {
   public void foo() {
      Runnable r = new Runnable() {
         @Override
         public void run() {
            System.out.println("Only way possible before lambdas!");
         }
      };
   }
}
```

This is usually called an "anonymous implementation of `Runnable`"; however, classes like this are not really anonymous—instead, the compiler will generate a class named something like `Scratch$1`, which is a genuine and usable Java class. Although the class name is not available to Java source code, the class can be found using that name and accessed reflectively and then used just like any other class.

A hidden class is not truly anonymous, either—it has a name that is available by directly invoking `getName()` on its Class object. This name can also show up in several other places, including diagnostic, JVM Tool Interface (JVMTI), or JDK Flight Recorder (JFR) events. However, hidden classes cannot be found using a class loader or in any way that regular classes can be found, including using reflection (e.g., via `Class.forName()`).

The intention is that hidden classes are named in a way that explicitly puts them in a different namespace from regular classes—the name has a sufficiently unusual form that it effectively makes the class invisible to all other classes.

The naming scheme exploits the fact that in the JVM classes typically have two forms of their name: the binary name (`com.acme.Gadget`), which is returned by calling `getName()` on a class object, and the internal form (`com/acme/Gadget`). Hidden classes are named in a way that does not fit this pattern. Instead, a name like `com.acme.Gadget/1234` would be returned by calling `getName()` on the class object of a hidden class. This is neither a binary name nor an internal form, and any attempt to make a regular class that matches this name will fail. Let's have a quick look at an example of how to create a hidden class:

```
var fName = "/Users/ben/projects/books/resources/Ch15/ch15/Concat.class";
var buffy = Files.readAllBytes(Path.of(fName));
```

```
var lookup = MethodHandles.lookup();
var hiddenLookup = lookup.defineHiddenClass(buffy, true);
var klazz = hiddenLookup.lookupClass();
System.out.println(klazz.getName());
```

One advantage of this naming scheme (and differentiating hidden classes in this way) is that they need not be subject to the usual vigorous scrutiny of the JVM's class loading mechanism. This fits with the overall design that hidden classes are intended for use by framework authors and others who need capabilities that go beyond the usual bullet-proof checks imposed on general Java classes.

NOTE Hidden classes were delivered as part of Java 15 and are not available in Java 11.

In the context of `Unsafe`, JEP 371 aims to deprecate the `defineAnonymousClass()` method from `Unsafe`, with the overall goal being to remove it in a future release. This is a purely internal change—there is no suggestion that the arrival of hidden classes will change the Java programming language in any way, at least initially. However, the implementations of classes like `LambdaMetaFactory`, `StringConcatFactory`, and other "flexible factory" methods could well be updated to use the new APIs.

Summary

- Java provides features for runtime introspection not easily available in languages like C++.
  - Reflection
  - Method handles
  - `invokedynamic`
  - `Unsafe`

Previous chapter

< 16 Advanced concurrent programming

Next chapter

18 Future Java >